# PGSL Assignment 10

## Pushing docker image to docker hub -

We have our website in a docker container to run it using Kubernetes pod we have to push it to the docker hub using this commands -

```
docker push <docker-username>/<docker-image>
```

Now, on the docker desktop, enable the Kubernetes option from settings.

## Create a Kubernetes Deployment for our Website -

**1. Create a website-deployment.yaml File for our Website**: In this YAML file, we will define the deployment for our website using your Docker image.

**website-deployment.yaml -**

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: website-deployment
spec:
  replicas: 1  # Number of instances we want to run
  selector:
    matchLabels:
      app: website
  template:
    metadata:
      labels:
        app: website
    spec:
      containers:
      - name: website
        image: yash581/website:latest  #our Docker image
        ports:
        - containerPort: 80  #our web server runs on port 80
```

```
        env:
        - name: MYSQL_HOST
          value: mysql-service  #Kubernetes service name for MySQL
        - name: MYSQL_USER
          value: root  # MySQL username
        - name: MYSQL_PASSWORD
          value: rootpassword  # MySQL password
        - name: MYSQL_DB
          value: mysql-pv  #our MySQL database name
```

**Apply the Deployment**: Now, run the following command to create the deployment in our Kubernetes cluster:

```
kubectl apply -f website-deployment.yaml
```

## Expose the Deployment with a Service

Once the deployment is created, we need to expose it using a Kubernetes service to make it accessible from outside the cluster.

**Create a website-service.yaml File**: We will create a NodePort service for our website, which will expose our website on a specific port of your local machine:

**website-service.yaml -**

```
apiVersion: v1
kind: Service
metadata:
  name: website-service
spec:
  selector:
    app: website  # Must match the label of the deployment
  ports:
    - protocol: TCP
```

```
    port: 80  # The port your web app listens on inside the pod
    targetPort: 80  # Same as container port
    nodePort: 30007  # Port exposed on our local machine (can be any port)
  type: NodePort
```

Run the following command to create the service -

```
kubectl apply -f website-service.yaml
```

Check that the service was created successfully -

```
kubectl apply -f website-service.yaml
```

We will see output like this -

```
NAME              TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
kubernetes        ClusterIP   10.96.0.1      <none>        443/TCP        8h
website-service   NodePort    10.98.96.43    <none>        80:30007/TCP   8h
```

## Create MySQL Deployment and Service -

We need a MySQL deployment and a service so that our website can connect to it. mysql-deployment.yaml defines the MySQL deployment with the necessary environment variables for MySQL (e.g., root password, database name).

**mysql-deployment.yaml -**

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql-deployment
spec:
  replicas: 1
  selector:
```

```
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
      - name: mysql
        image: mysql:5.7  # Use the official MySQL Docker image
        env:
        - name: MYSQL_ROOT_PASSWORD
          value: "rootpassword"  # Set a strong password
        ports:
        - containerPort: 3306
        volumeMounts:
        - name: mysql-storage
          mountPath: /var/lib/mysql  # MySQL data location
      volumes:
      - name: mysql-storage
        persistentVolumeClaim:
          claimName: mysql-pvc  # Link to the PVC
```

Now mysql-pv-pvc.yaml defines a persistent volume claim (PVC) that ensures MySQL's data is stored persistently.

**mysql-pv-pvc.yaml -**

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: mysql-pv
spec:
```

```
  capacity:
    storage: 1Gi  # Adjust based on your storage requirements
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: /mnt/data/mysql  # Path to your host directory for persistence
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mysql-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

Now that we have defined the MySQL deployment, service, and persistent volume, apply them to Kubernetes using the following commands:

```
kubectl apply -f k8s/mysql-pv-pvc.yaml
kubectl apply -f mysql-deployment.yaml
```

Verify that all our pods are running:

```
kubectl get pods
```

We will get output like this -

```
NAME                                  READY   STATUS    RESTARTS   AGE
mysql-deployment-6f8ff997f4-pf8rs     1/1     Running   0          8h
website-deployment-775db599c-k7m52    1/1     Running   0          8s
```

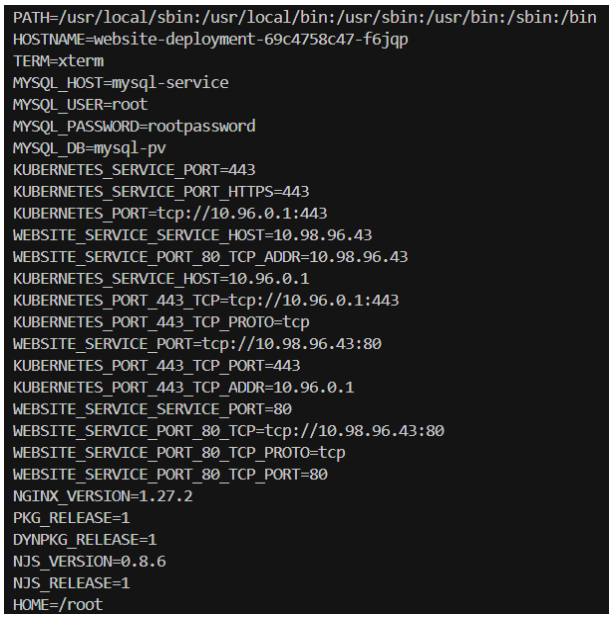
Now we can access our website on http://localhost:30007


---

We can verify if MySql is connected to our website by checking the environment variables of our website pod using the command:

```
kubectl exec -it <kubernates-pod-name> -- env
```

We will see the output like this:



Here, we can see my-sql dataset is connected to our website.

To stop the service, we can use command like -

```
kubectl scale deployment <service-name> --replicas=0
```

To start it again, we can use:

```
kubectl scale deployment <service-name> --replicas=1
```